

Laboratorio di Informatica 1

Corso di Laurea in Fisica

Federico Nati, Uni. Milano-Bicocca, 2021

Lezione 3, 24 marzo 2021

ARGOMENTI

Abbiamo visto in precedenza:

- Architettura di un calcolatore
- CPU, memoria RAM, disco fisso
- Codice sorgente, linguaggio macchina, compilatore
- Differenza tra file sorgente, file eseguibile compilato e processo in memoria
- Sistema operativo
- File system
- Strutture di controllo in C (selezione, iterazione)
- Esempi di algoritmi di ordinamento e confronto tra costi computazionali

Oggi vedremo:

- Richiamo sulla struttura generale di un programma in C
- Ruolo dei commenti e la scelta dei nomi
- Un'altra struttura di controllo di iterazione: il ciclo `for`
- Un primo tipo di dato strutturato: l'array
- Le funzioni
- Campo di visibilità (scope) degli identificatori

Appendice:

- Riga di comando
- Canali standard (stdin e stdout)
- Chiarimenti su `scanf`; concetto di buffer di input

Struttura generale di un programma in C

Ruolo dei commenti

STRUTTURA GENERALE DI UN PROGRAMMA IN C

Direttive del preprocessore

Ci permettono di utilizzare alcune funzionalità come la possibilità di scrivere testo nella finestra del terminale

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main (int argc, char * argv[])
{
    /* qui il vostro codice */
    exit(EXIT_SUCCESS);
}
```

"Entry point"

Dove inizia l'esecuzione del programma

Funzione *exit*

Segnala al sistema operativo che il programma ha terminato l'esecuzione con successo

Parentesi graffe

Sono usate per indicare dove inizia e finisce il main.
In generale servono a delimitare un blocco di codice

STRUTTURA GENERALE DI UN PROGRAMMA IN C

```c
/**
* Commento iniziale
*/

#include <stdio.h>
#include <stdlib.h>

/* Altre direttive */

/* Dichiarazione delle variabili globali */

/* Definizione delle funzioni */

int main (int argc, char * argv[]) {
    /* Dichiarazione delle variabili locali */

    /* Corpo della funzione main */

    exit (EXIT_SUCCESS);
}
```

Ricordiamoci che è un testo, quindi deve essere leggibile

I commenti devono essere informativi ma non troppo, mai banali

I nomi scelti per le variabili, dati, funzioni, devono essere ben pensati

Inoltre i nomi scelti hanno delle regole bene precise da rispettare

COMMENTI

```
int x; // Commento su una singola linea

/* Commento su linee multiple.
Anche questo viene ignorato
Fino a qui */

/*
* Commento su linee multiple.
* Anche questo viene ignorato
* Fino a qui. Lo stesso di prima ma
* formattato in modo diverso
*/
```

- Commentare all'inizio del programma per spiegare cosa deve fare
- Evitare commenti ovvi o che non aggiungono nulla alla lettura del semplice codice. Esempio di commento **da non fare:**

```
x = x + 1; // incremento x di uno
```

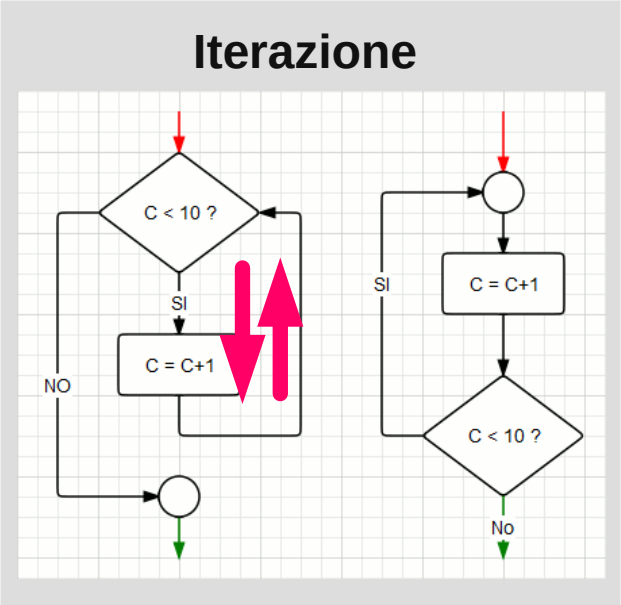
- Spiegare perché si fa qualcosa
- Immaginate di condividere il codice con altri, o di dover tornare a leggerlo a distanza di mesi o anni

Iterazioni: il ciclo `for`

ITERAZIONE

Abbiamo già visto che si possono eseguire delle iterazioni sullo stesso blocco di codice usando il ciclo **while** (o do, while)

In C e in generale in molti linguaggi di programmazione esiste anche un'altra forma di ripetere le stesse operazioni, cioè un'altra istruzione per costruire la struttura di controllo dette iterazione: il ciclo **for**



ITERAZIONE: CICLO WHILE CON VARIABILE INDICE

Variabile “indice”

Dichiariamo ed inizializziamo una variabile che conta il numero di volte che abbiamo eseguito il ciclo

```
int i = 0;
while (i < 10) {
    /* qui mettiamo il nostro codice */
    i = i + 1;
}
```

Condizione di uscita

Controlliamo se proseguire il ciclo

Incremento della variabile indice

Aumentiamo il valore della variabile indice

Variabile “indice”

La possiamo definire come prima parte del ciclo for. questa codice viene eseguito prima della prima iterazione

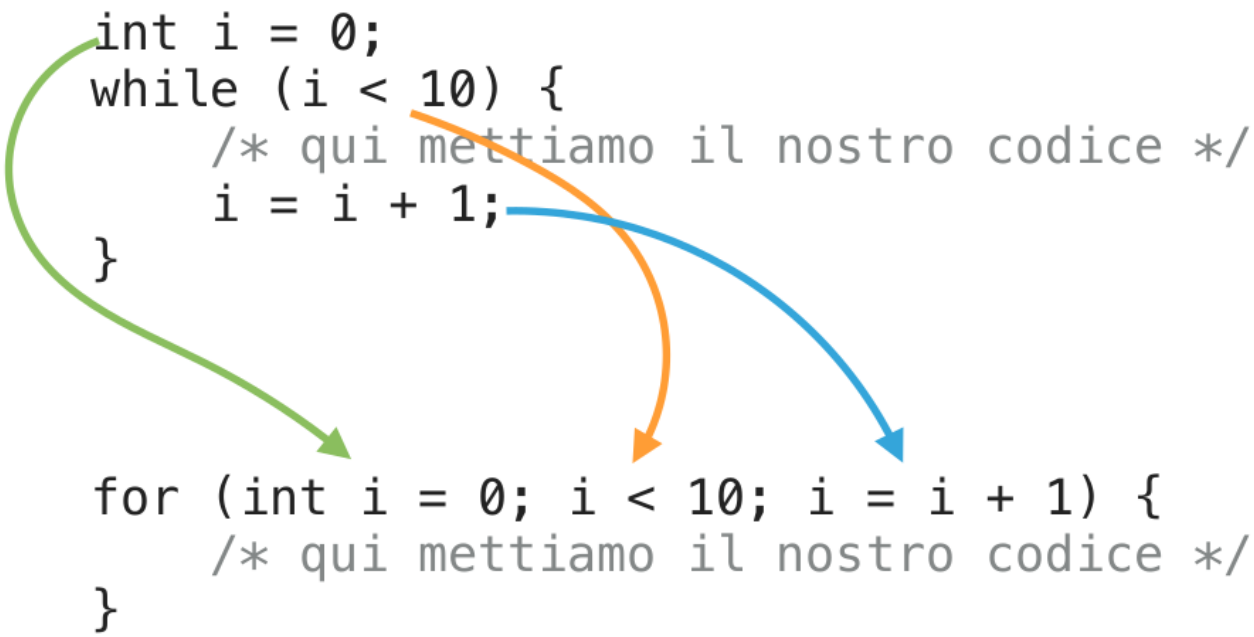
Condizione di uscita

Viene controllata all’inizio di ogni ciclo. Se falsa si esce dal ciclo

```
for (int i = 0; i < 10; i = i + 1) {
    /* qui mettiamo il nostro codice */
}
```

Incremento della variabile indice

Aumentiamo il valore della variabile indice, è l’ultima istruzione eseguita



Nota: vedremo più avanti il concetto di campo di visibilità. Per ora qui limitiamoci a dire che la “`i`”, e in generale la variabile usata nell’intestazione del `for`, non è utilizzabile una volta esaurito il ciclo for stesso, cosa che invece è possibile fare con il ciclo while mostrato nell’esempio.

ESEMPIO WHILE E FOR

```c
#include <stdio.h>
#include <stdlib.h>

int N = 7;
int i = 0;
int i_quadro;
long int SOMMA = 0;

void main() {

    while (i <= N) {
        i_quadro = i*i;
        SOMMA = SOMMA + i_quadro;
        i += 1;
    }

    printf("%ld\n", SOMMA);

    exit(0);
}
```

```c
#include <stdio.h>
#include <stdlib.h>

int N = 7;
int i;
int i_quadro;
long int SOMMA = 0;

void main() {

    for (i=1; i <= N; i++) {
        i_quadro = i*i;
        SOMMA = SOMMA + i_quadro;
    }

    printf("%ld\n", SOMMA);

    exit(0);
}
```

CICLO FOR

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N		i		i^2		SOMMA		
7		1				0		

```
int N = 7;
int i;
int i_quadro;
long int SOMMA = 0;
```

```
for (i=1; i <= N; i++) {
    i_quadro = i*i;
    SOMMA = SOMMA + i_quadro;
}
```

CICLO FOR

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N	i		i^2			SOMMA		
7	1					0		

```
for (i=1; i <= N; i++) {
    i_quadro = i*i;
    SOMMA = SOMMA + i_quadro;
}
```

CICLO FOR

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N		i		i^2		SOMMA		
7		1				0		
7		2		1		1		

```
for (i=1; i <= N; i++) {
    i_quadro = i*i;
    SOMMA = SOMMA + i_quadro;
}
```

CICLO FOR

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N		i		i^2		SOMMA		
7		1				0		
7		2		1		1		

```
for (i=1; i <= N; i++) {
    i_quadro = i*i;
    SOMMA = SOMMA + i_quadro;
}
```

CICLO FOR

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N	i	i^2	SOMMA					
7	1		0					
7	2	1	1					

```
for (i=1; i <= N; i++) {
    i_quadro = i*i;
    SOMMA = SOMMA + i_quadro;
}
```

CICLO FOR

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N		i		i^2		SOMMA		
7		1				0		
7		2		1		1		
7		3		4		5		

```
for (i=1; i <= N; i++) {
    i_quadro = i*i;
    SOMMA = SOMMA + i_quadro;
}
```

CICLO FOR

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N		i		i^2		SOMMA		
7		1				0		
7		2		1		1		
7		3		4		5		
7		4		9		1	4	
7		5		1	6	3	0	
7		6		2	5	5	5	
7		7		3	6	9	1	
7		8		4	9	1	4	0

$f(7) = 140$

```
for (i=1; i <= N; i++) {
    i_quadro = i*i;
    SOMMA = SOMMA + i_quadro;
}
```

CICLI FOR ANNIDATI

Ciclo esterno, viene eseguito 3 volte

```
for (int i = 1; i <= 3; i = i + 1) {
  for (int j = 1; j <= 3; j = j + 1) {
    int prodotto = i * j;
    printf("%d * %d = %d\n", i, j, prodotto);
  }
}
```

Ciclo interno, viene eseguito 3 volte
per ogni esecuzione del ciclo esterno,
quindi 9 volte in totale

Output:

```
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
```

Array di dati

TIPI STRUTTURATI: ARRAY

- I tipi di **dati strutturati** sono aggregazioni di dati, organizzati o legati secondo una qualche relazione
- Il più semplice tipo strutturato è l'**array** (**vettore** in italiano, qui usati come sinonimi)
- Sono estremamente utili perché spesso i dati che ci interessa trattare sono organizzati in liste, vettori, tabelle
- Quindi è utile avere un solo nome di variabile che indichi una successione di molti elementi
- Gli array sono composti da elementi dello stesso tipo individuati da un **indice** che parte da 0, e sono distribuiti in modo **consecutivo** nella RAM

- Per esempio se avessimo 1000 campionamenti di una osservabile fisica raccolta in laboratorio, invece di scrivere in C

```
float dato0, dato1,,,, dato999;
```

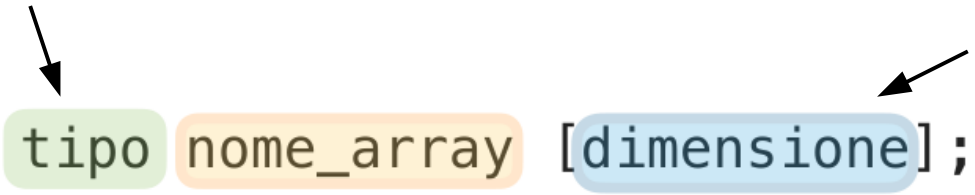
possiamo scrivere:

```
float dati[1000];
```

DICHIARAZIONE DI UN ARRAY

Tipo delle celle

Possono essere interi (short, int, long, ...), numeri a virgola mobile (float, double), caratteri (char), etc.



Dimensione

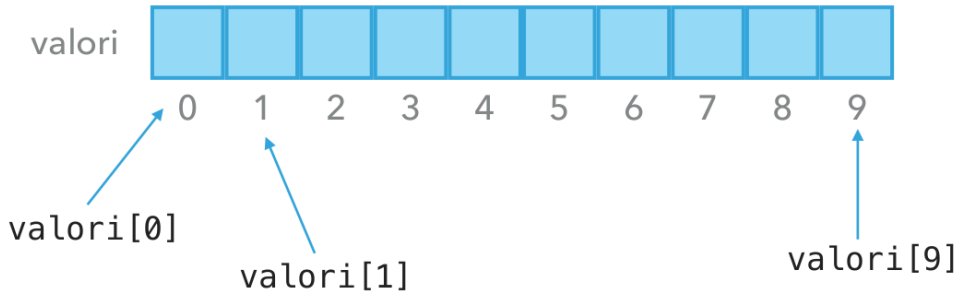
Numero di celle (un valore intero positivo)

Nome della variabile

Si riferisce all'intero array (il "contenitore")

ARRAY, INDICI E MEMORIA

- Gli elementi di un array in C sono tutti dello stesso tipo
- Organizziamo N valori dandogli un “nome unico”, cioè un identificatore per il vettore di dati
- A ciascun array viene assegnato un blocco di memoria della lunghezza giusta per contenerne tutti gli elementi
- Se per esempio un singolo elemento fosse di un tipo che richiede 4 byte (4 celle di RAM) per essere rappresentato, e l'array fosse di 1000 elementi, allora verrebbe allocato uno spazio di memoria pari a 4 byte * 1000 elementi = 4 kB
- Gli elementi di un array sono disposti in modo consecutivo in memoria
- Gli array non “sanno”, o meglio non contengono l'informazione della loro dimensione: dobbiamo ricordarcela a parte e, se ci serve, salvarla in una variabile
- Accediamo ai singoli elementi con il nome del vettore e l’indice di elemento, dove il primo elemento ha indice 0, e l'ultimo elemento ha indice N-1 (se N è il numero totale di elementi). Per esempio `valori[4]` si riferisce al quinto elemento all’interno dell'array valori:



DISPOSIZIONE DEGLI ARRAY IN MEMORIA

Tipo delle celle

Numero di celle

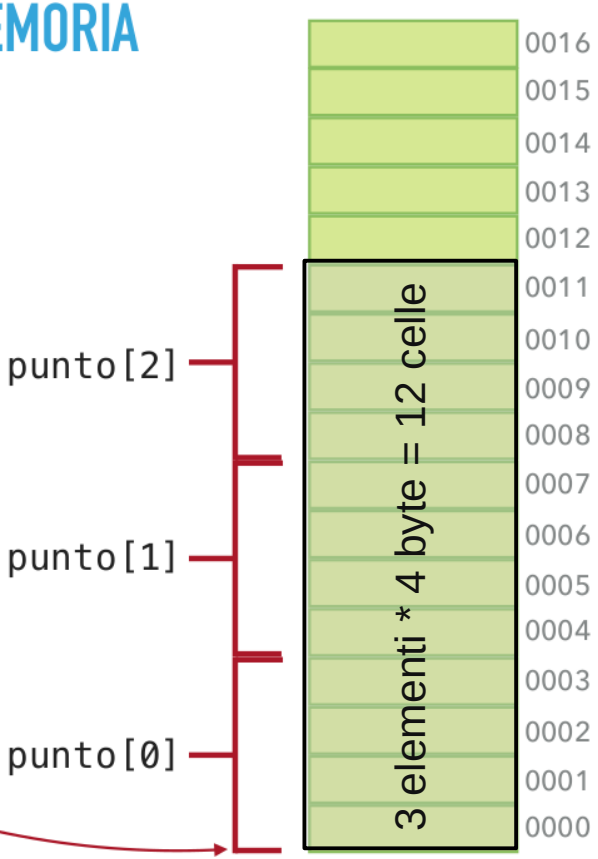
```
float punto[3];
```

Nome dell'array

Ma cosa è "punto"?

Un indirizzo di memoria
che dice dove inizia l'array

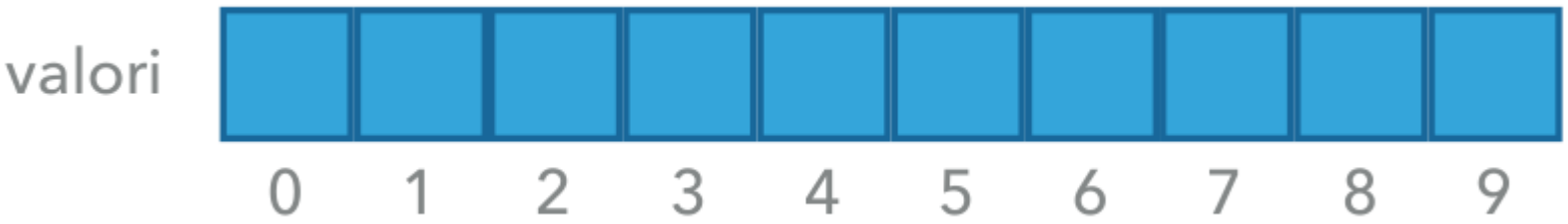
punto



```
int valori [10];

valori[2] = 4;
valori[5] = 2;

valori[valori[2]] = 3;
```



```
int valori [10];

valori[2] = 4;
valori[5] = 2;

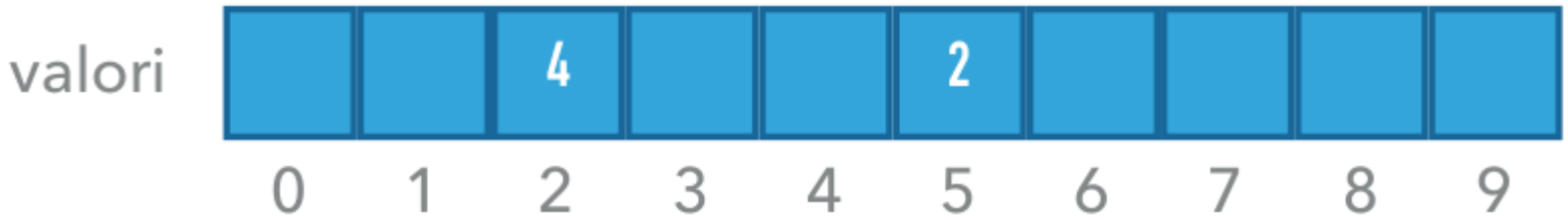
valori[valori[2]] = 3;
```



```
int valori [10];

valori[2] = 4;
valori[5] = 2;

valori[valori[2]] = 3;
```



```
int valori [10];
int indice = 4;

valori[2] = 4;
valori[5] = 2;


valori[indice] = 3;
```

Nota: l'indice deve essere un intero. Può anche essere un'espressione da valutare, purché di risultato intero.



OPERAZIONI COMUNI CON GLI ARRAY

Ogni elemento di un vettore è una variabile, che si può usare come una qualsiasi variabile dello stesso tipo:

```
val[5] = x+3;

val[i+1] = val[i]*2;

temperature[0] = -3.4;

sigla[2] = 'a';

if ( val[i] < 0 ) ...
```

```
int quadrati[10];

for (int i = 0; i < 10; i = i + 1) {
    quadrati[i] = i * i;
}
```

Popolare gli elementi dell'array con dei valori

```
for (int i = 0; i < 10; i = i + 1) {
    printf("%d * %d = %d\n", i, i, quadrati[i]);
}
```

Stampare gli elementi dell'array

Per esercizio: scrivere un codice che copi i valori degli elementi di un vettore in un altro vettore, impostando un'iterazione nella quale copio un elemento per volta nella posizione omologa.

OPERAZIONI COMUNI CON GLI ARRAY

Scambiare di posto i valori di due elementi

```
temp = val[0];
val[0] = val[1];
val[1] = temp;
```

Attraversare tutto l'array per eseguire un'operazione, per es. stamparne gli elementi, o cambiarne il valore, ecc.

```
for (i=0; i<N; i++) {
        val[i] = -val[i];
}
```

INIZIALIZZAZIONE DI UN ARRAY

- Quando un array viene dichiarato, viene allocato lo spazio in memoria per contenerlo
- Tuttavia le celle di memoria così assegnate potrebbero contenere informazioni preesistenti, o essere in uno stato indefinito
- Finché non scriviamo noi quali valori devono essere scritti in tali locazioni di memoria, non possiamo sapere che valori otteniamo se proviamo a leggerle
- Pertanto dobbiamo prestare molta attenzione a non leggere il valore degli elementi prima di assegnarne i valori, o prima di averli inizializzati
- **Attenzione**: l'identificatore (nome) di un array non è una variabile, per cui non possiamo scrivere una cosa del tipo

NO
```
int myArray[10];
myArray = 10;
```

NO
```
int v[10], w[10];
w = v;
```

- Un modo di impostare i valori iniziali di un array è quello di inizializzarne tutti i valori a 0, così

```
int myArray[10] = { 0 };
```

- Oppure specificando i singoli valori da inizializzare:

```
int myArray[] = {12, 34, 1, 0, 34, 1};
```

In questo caso si può omettere la lunghezza.
Se si esplicita, deve essere coerente con i dati

ARRAY MULTIDIMENSIONALI

- Oltre a “vettori” (array monodimensionali), possiamo avere matrici, cioè array bi- o multidimensionali
- L’accesso agli elementi di un array a 2 dimensioni necessita di due indici, a tre dimensioni di tre indici, e così via
- Per il resto si comportano in modo simile agli array di una singola dimensione

ARRAY MULTIDIMENSIONALI

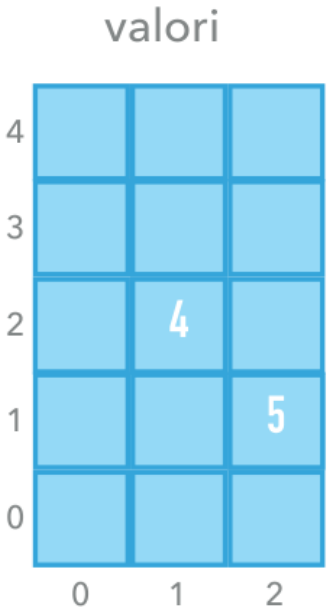
```
int valori [5][3];
```

Array di 5 righe e 3 colonne

L'accesso agli elementi richiede due indici:

```
valori[2][1] = 4;

valori[1][2] = 5;
```

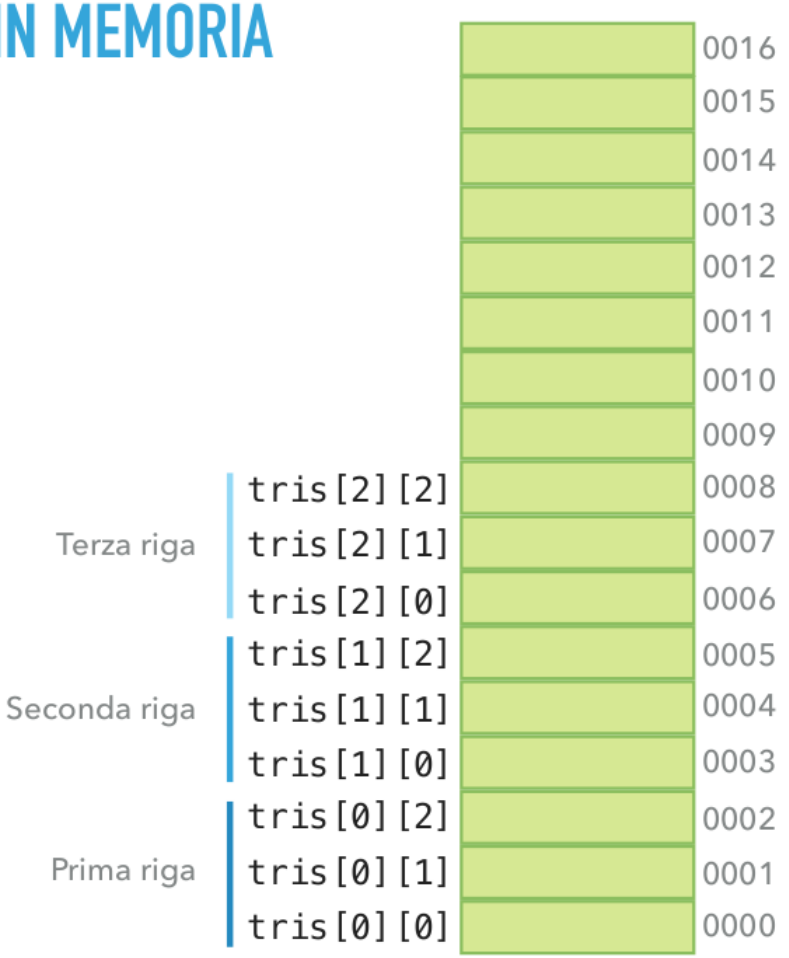


ARRAY MULTIDIMENSIONALI IN MEMORIA

Tipo delle celle Numero di righe

```
char tris [3][3];
```

Nome dell'array Numero di colonne



Funzioni

```c
/**
* Commento iniziale
*/

#include <stdio.h>
#include <stdlib.h>

/* Altre direttive */

/* Dichiarazione delle variabili globali */

/* Definizione delle funzioni */

int main (int argc, char * argv[]) {
    /* Dichiarazione delle variabili locali */

    /* Corpo della funzione main */

    exit (EXIT_SUCCESS);
}
```

Abbiamo già parlato di funzioni, senza però dire cosa sono

la funzione main() è il primo esempio che avete visto

Avete già usato delle funzioni, per esempio scanf() e printf()

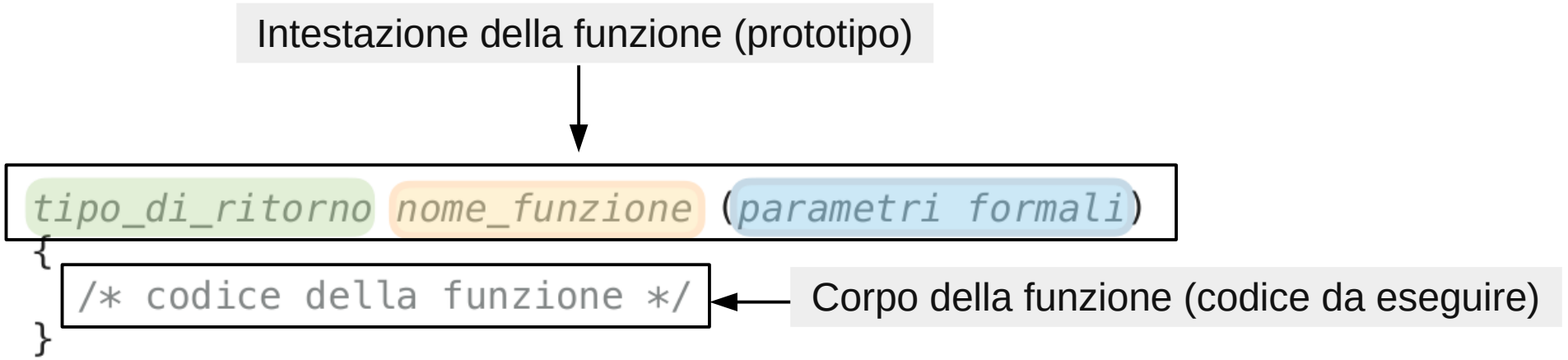
Nella struttura generale del codice è indicato uno spazio per scrivere altre funzioni

FUNZIONI

- Il termine *funzione* lo avete già sentito in matematica, es. f(x). Essa ha un dominio, una o più variabili indipendenti, ovvero lo spazio dei valori di ingresso e un codominio, ovvero lo spazio dei valori di uscita. La sua definizione specifica i calcoli che devono essere eseguiti sulla variabile indipendente affinché si ottenga il valore (o i valori) di uscita.
- Nella programmazione, **una funzione è una sezione di un programma che svolge un compito specifico, riceve degli argomenti in ingresso e produce un risultato (o un effetto).**
- Quando scriviamo del codice vorremmo riutilizzare alcune idee, algoritmi, operazioni, specificando di volta in volta dati di ingresso diversi, e intercettarne l'uscita per svolgere altre operazioni successive
- Oppure vorremmo ripetere alcune procedure più volte, in punti diversi del codice
- In fase di progettazione, vorremmo scrivere quali sono, appunto, le funzioni di parti del codice e come possono "chiamarsi" l'una con l'altra, ragionando successivamente su come realizzarle in pratica (passare dall'astratto al concreto, descrivendo una visione generale dei problemi per affrontare e rifinire i dettagli in seguito è uno stile di ragionamento chiamato tipo ***top-down***)
- Come avviene per le funzioni in matematica, una funzione in programmazione può chiamarne un'altra, es: f(g(x)), o anche sé stessa (funzioni ricorsive)

FUNZIONI

- Funzione: sezione di un programma che svolge un compito specifico, riceve degli argomenti in ingresso e produce un risultato (o un effetto)
- Servono anche a organizzare il codice, a renderlo più leggibile, a rendere più chiaro e semplice il riutilizzo
- Una funzione ha dei dati o parametri di input, detti argomenti, un corpo, cioè la porzione di codice che ne definisce le operazioni, un tipo, che definisce il tipo di dati che vengono restituiti.
- Se ha un'istruzione che restituisce dati in output, si chiama funzione, se invece svolge operazioni senza restituire dati, si dice procedura (e di solito ha il tipo void)
- Un programma in C consiste in una serie di definizioni di funzioni (oltre ad altri elementi). Il nome `main` denota una funzione speciale, che deve essere presente in ogni programma e dalla quale prende avvio l'esecuzione
- Cerchiamo di mantenere il `main()` il più semplice possibile, come se fosse una sorta di sommario delle azioni da svolgere, e spostare i dettagli in funzioni esterne. In altre parole, cerchiamo di **articolare il codice in funzioni**
- Da una funzione è possibile chiamare (o invocare) altre funzioni, specificando i valori degli argomenti; al termine dell'esecuzione della funzione chiamata, la funzione chiamante riceve il valore prodotto dalla chiamata.



Il corpo della funzione contiene le istruzioni da eseguire quando la funzione viene chiamata. In particolare, deve contenere un'istruzione speciale, chiamata `return()`, che segnala la fine dell'esecuzione e specifica il valore prodotto dalla funzione

Tipo di ritorno

Indica che tipo di valori produce la funzione:
interi, float, etc.
Se non produce nulla il tipo di ritorno è void

Nome della funzione

Dice come chiameremo la funzione
nel nostro codice

```
tipo_di_ritorno nome_funzione (parametri formali)
{
  /* codice della funzione */
}
```

Parametri formali

Un elenco di dichiarazioni di variabili: indicano che input riceve la funzione
Possono essercene 0, 1 o più di uno

ESEMPIO

```c
#include <stdio.h>
#include <stdlib.h>

#define M_PI 3.14159265358979323846

double circonferenza_da_raggio (double raggio) {
    /* calcola la circonferenza di un cerchio in funzione del suo raggio */

    double circ = 2 * M_PI * raggio;
    return(circ);

}

void main() {
    double circonferenza;
    double raggio[] = {1, 2, 3, 4, 5, 6, 7 ,8 ,9, 10};


    for (int i=0; i<10; i++) {
        circonferenza = circonferenza_da_raggio(raggio[i]);
        printf("Raggio: %1.2lf\tCirconferenza: %1.2lf\n", raggio[i], circonferenza);
    }

    exit(0);
}
```

ESEMPIO

```c
#include <stdio.h>
#include <stdlib.h>

#define M_PI 3.14159265358979323846

double circonferenza_da_raggio (double raggio) {
    /* calcola la circonferenza di un cerchio in funzione del suo raggio */

    double circ = 2 * M_PI * raggio;
    return(circ);

}

void main() {
    double circonferenza;
    double raggio[] = {1, 2, 3, 4, 5, 6, 7 ,8 ,9, 10};


    for (int i=0; i<10; i++) {
        circonferenza = circonferenza_da_raggio(raggio[i]);
        printf("Raggio: %1.2lf\tCirconferenza: %1.2lf\n", raggio[i], circonferenza);
    }

    exit(0);
}
```

```
Raggio: 1.00    Circonferenza: 6.28
Raggio: 2.00    Circonferenza: 12.57
Raggio: 3.00    Circonferenza: 18.85
Raggio: 4.00    Circonferenza: 25.13
Raggio: 5.00    Circonferenza: 31.42
Raggio: 6.00    Circonferenza: 37.70
Raggio: 7.00    Circonferenza: 43.98
Raggio: 8.00    Circonferenza: 50.27
Raggio: 9.00    Circonferenza: 56.55
Raggio: 10.00   Circonferenza: 62.83
```

ARGOMENTI FORMALI E REALI, RECORD DI ATTIVAZIONE

- Gli identificatori degli argomenti nell'intestazione di una funzione sono chiamati **argomenti formali**

- I valori specificati nella chiamata di una funzione sono chiamati **argomenti reali**

- In questo contesto "argomento" e "parametro" sono sinonimi. Un argomento reale può essere una variabile o un valore specifico.
  Ad esempio: `r = f(a, 2*b, 0.2);`

- Si dice che la funzione restituisce un valore alla funzione chiamante (che può usarlo in un assegnamento, come nell'esempio precedente, in una istruzione di stampa, o in altro modo)

- Le variabili utilizzate da una funzione vivono in uno spazio di memoria che viene creato ogni volta che si chiama la funzione, e che viene cancellato quando quella chiamata termina. Tale spazio di memoria si chiama **record di attivazione** di una funzione.

COMPOSIZIONE DI FUNZIONI

- Le chiamate delle funzioni possono essere composte.

- Supponiamo di avere definito due funzioni con queste intestazioni:

```
double f (int n, double x)
int g (double x, double y)
```

- In una terza funzione, nella quale è stata dichiarata una variabile p, di tipo int, possiamo scrivere l'assegnamento:

```
p = g ( f (2, 0.5), -1.25);
```

- Per valutare l'espressione a destra dell'uguale, si dovrà prima chiamare la funzione f, con argomenti reali 2 e 0.5; il risultato diventerà il primo argomento reale della chiamata alla funzione g.

RETURN MULTIPLI

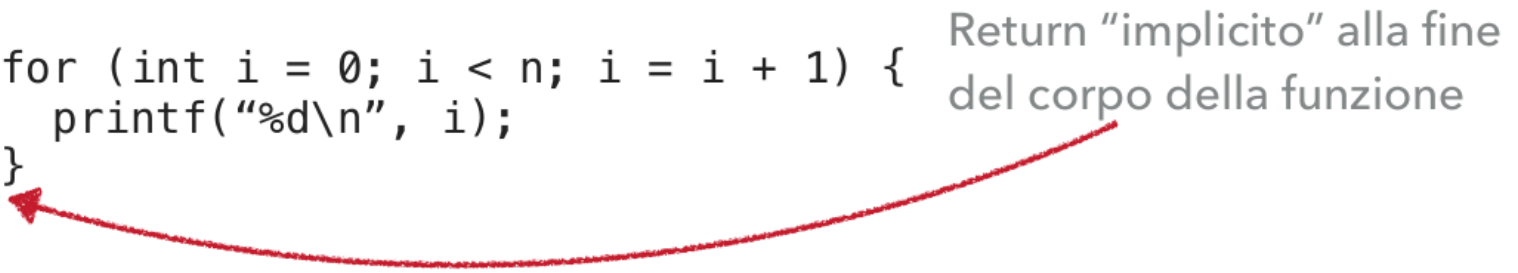
```c
 /* Funzione max
* Restituisce il valore massimo fra i tre valori ricevuti
* come argomenti.
*/

int max (int a, int b, int c)
{
    if ( a < b ) {
        if ( b < c )
            return c;
        else
            return b;
    }
    else {
        if ( a < c )
            return c;
        else
            return a;
    }
}
```

FUNZIONI CHE NON RESTITUISCONO VALORI (PROCEDURE)

```
void stampa_primi_n(int n)
{
  for (int i = 0; i < n; i = i + 1) {
    printf("%d\n", i);
  }
  return;
}
```

```
void stampa_primi_n(int n)
{
  for (int i = 0; i < n; i = i + 1) {
    printf("%d\n", i);
  }
}
```



PASSAGGIO DI ARRAY COME ARGOMENTI DI UNA FUNZIONE

- Gli argomenti delle funzioni visti fin qui vengono copiati nei parametri reali, e quindi essa **lavora su delle copie dei valori**

- Questo resta vero per i singoli elementi di un vettore, per es.:

```
x = max(5, v[3]);
```

- Tuttavia con gli array succede una cosa **diversa**, capiremo meglio più avanti il motivo

- Per ora va immagato che l'identificatore di un array sia l'**indirizzo** della posizione in memoria in cui l'array "inizia"

PASSAGGIO DI ARRAY COME ARGOMENTI DI UNA FUNZIONE

- L'identificatore rappresenta la posizione in memoria in cui l'array "inizia"
- La dimensione del vettore non è "attaccata" al vettore, ce la dobbiamo ricordare e passare alla funzione
- Quindi per passare l'array a una funzione e usarlo scriviamo per esempio:

```
void raddoppia(int v[], int n) {
    int i = 0;
    for ( i=0; i < n; i++ ) {
        v[i] = 2*v[i];
    }
    return;
}

int main (int argc, char *argv[]) {
    int w[] = { 3, -5, 1, 0 };

    ...
    raddoppia( w, 4 );
    ...
}
```

Notiamo che questa funzione è di tipo void e non restituisce alcun valore. Essa infatti opera direttamente sui valori presenti all'**indirizzo** del vettore originario, visto dal main, e **non crea alcuna copia**.

Passiamo il vettore indicandone **tipo**, **identificatore** seguito da **parentesi quadre**, senza il numero di elementi, che invece passiamo come **secondo parametro** (int n in questo caso).

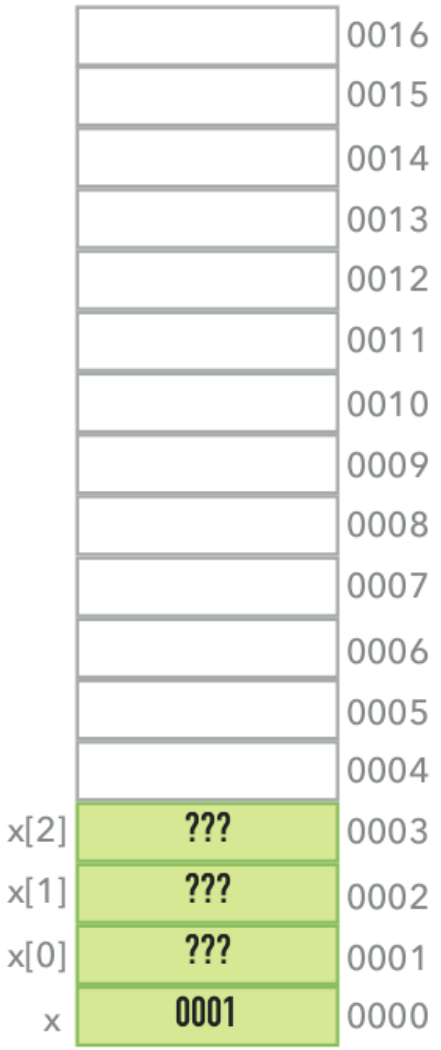
ARRAY E RECORD DI ATTIVAZIONE

```c
void riempi_array(int a[], int len) {
    for (int i = 0; i < len; i = i + 1) {
    a[i] = 5;
    }
}

int main(int argc, char * argv[]) {
    int x[3];
    riempi_array(x, 3);
    exit(EXIT_SUCCESS);
}
```

```c
void riempi_array(int a[], int len) {
    for (int i = 0; i < len; i = i + 1) {
    a[i] = 5;
    }
}

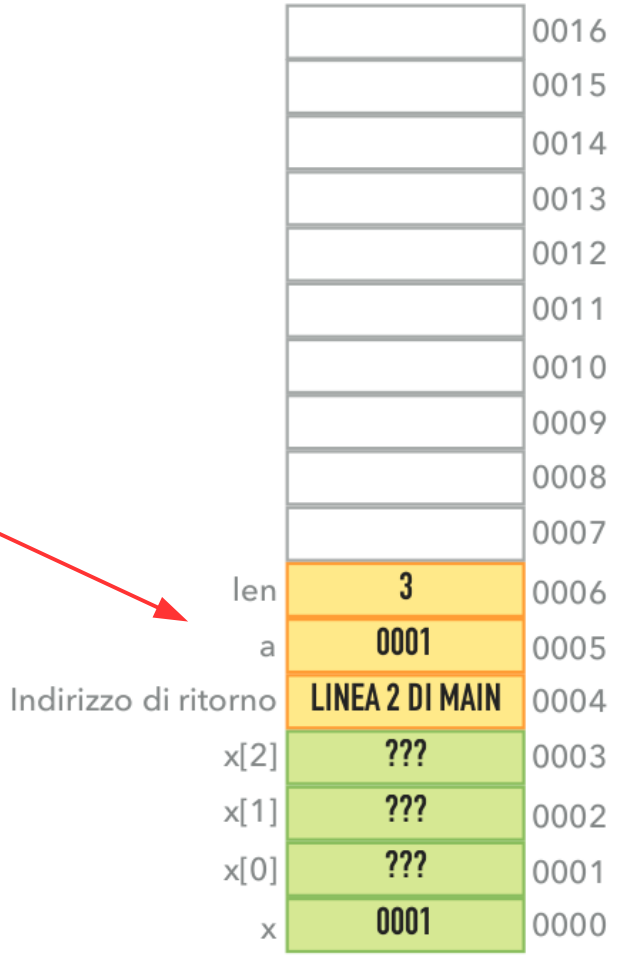
int main(int argc, char * argv[]) {
    int x[3];
    riempi_array(x, 3);
    exit(EXIT_SUCCESS);
}
```



ARRAY E RECORD DI ATTIVAZIONE

```c
void riempi_array(int a[], int len) {
    for (int i = 0; i < len; i = i + 1) {
    a[i] = 5;
    }
}

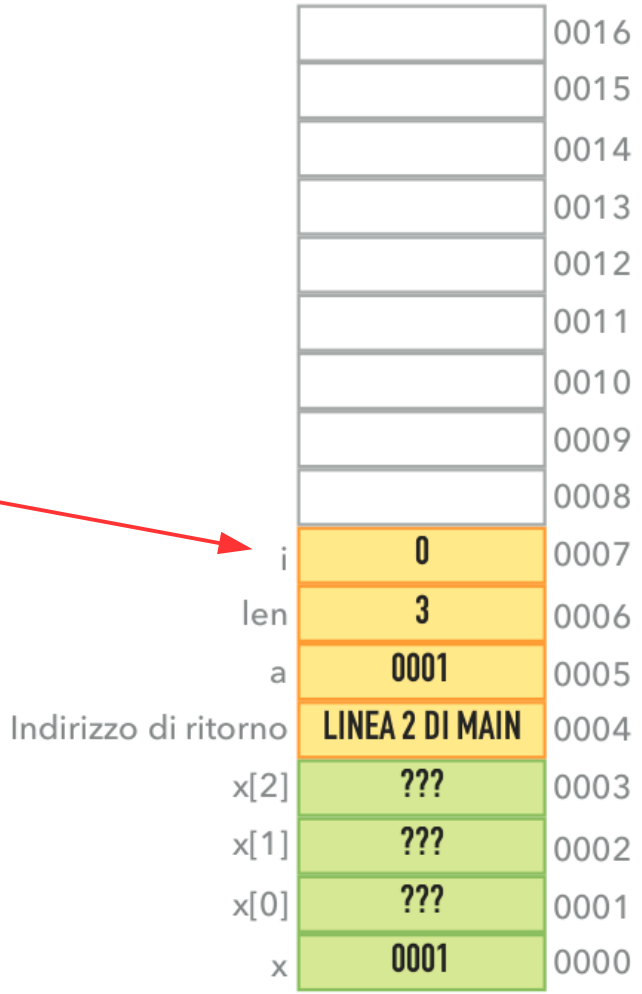
int main(int argc, char * argv[]) {
    int x[3];
    riempi_array(x, 3);
    exit(EXIT_SUCCESS);
}
```



ARRAY E RECORD DI ATTIVAZIONE

```c
void riempi_array(int a[], int len) {
    for (int i = 0; i < len; i = i + 1) {
    a[i] = 5;
    }
}

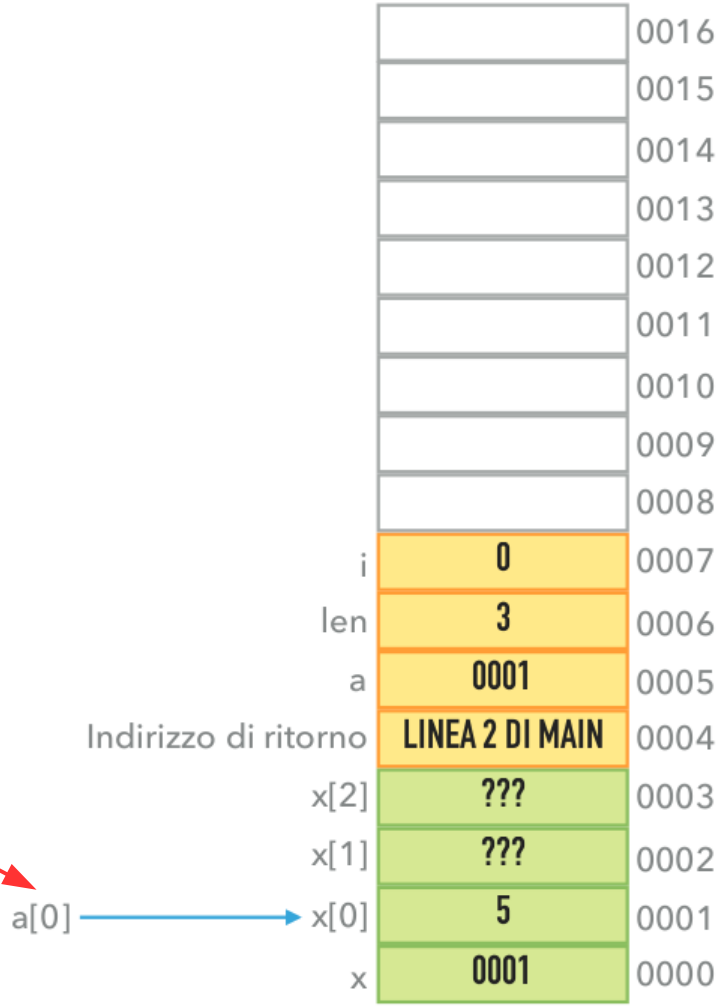
int main(int argc, char * argv[]) {
    int x[3];
    riempi_array(x, 3);
    exit(EXIT_SUCCESS);
}
```



ARRAY E RECORD DI ATTIVAZIONE

```c
void riempi_array(int a[], int len) {
    for (int i = 0; i < len; i = i + 1) {
    a[i] = 5;
    }
}

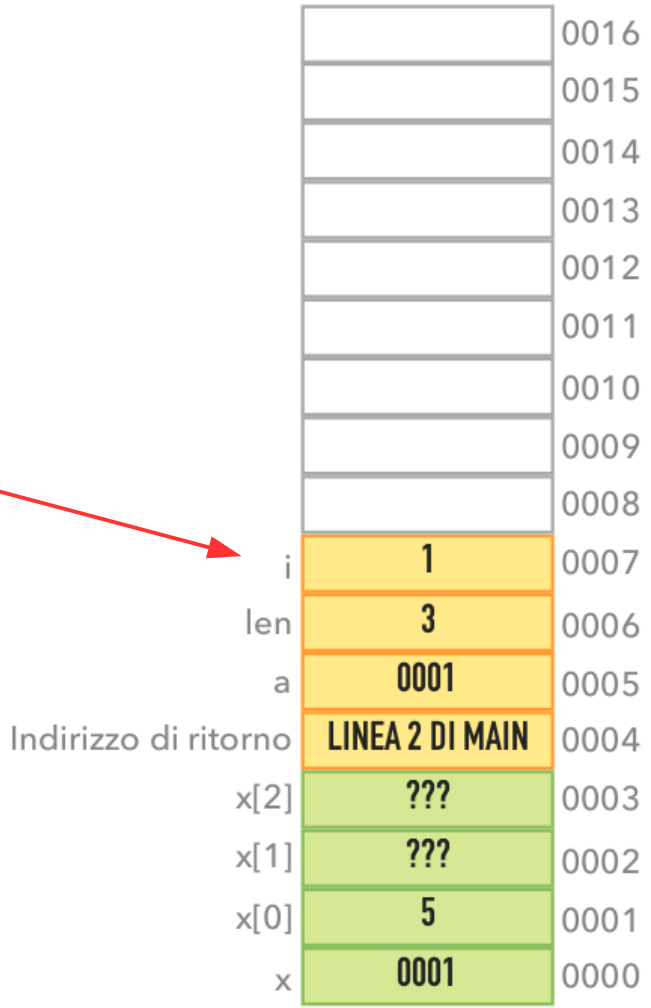
int main(int argc, char * argv[]) {
    int x[3];
    riempi_array(x, 3);
    exit(EXIT_SUCCESS);
}
```



ARRAY E RECORD DI ATTIVAZIONE

```c
void riempi_array(int a[], int len) {
    for (int i = 0; i < len; i = i + 1) {
    a[i] = 5;
    }
}

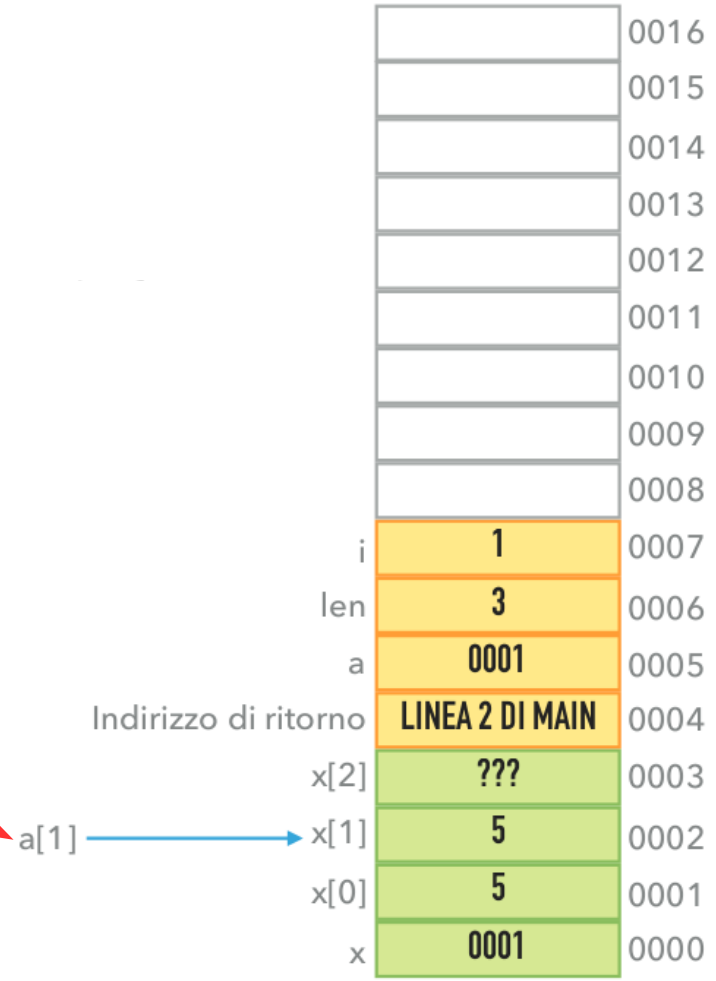
int main(int argc, char * argv[]) {
    int x[3];
    riempi_array(x, 3);
    exit(EXIT_SUCCESS);
}
```



ARRAY E RECORD DI ATTIVAZIONE

```c
void riempi_array(int a[], int len) {
    for (int i = 0; i < len; i = i + 1) {
    a[i] = 5;
    }
}

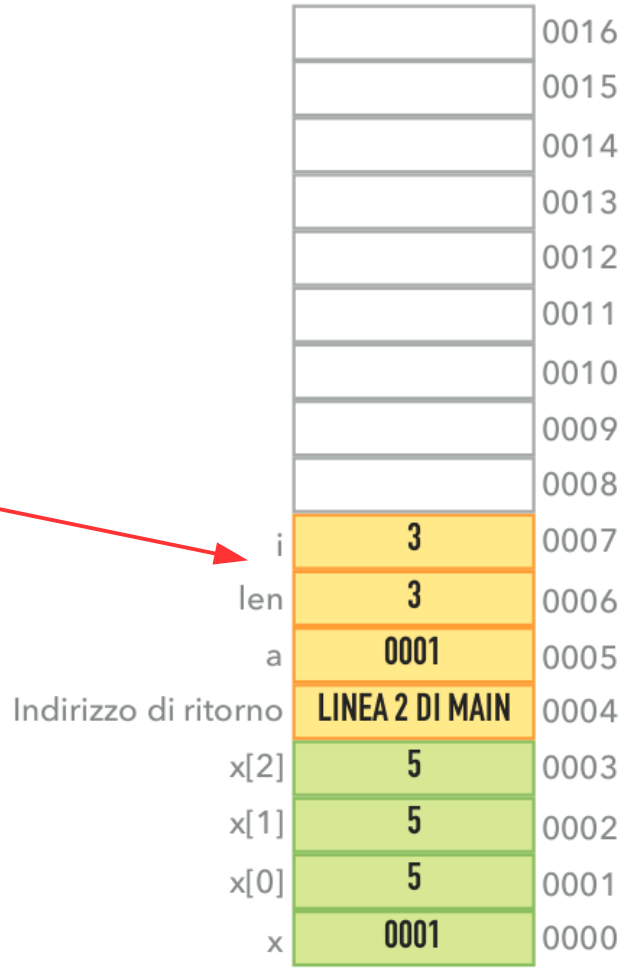
int main(int argc, char * argv[]) {
    int x[3];
    riempi_array(x, 3);
    exit(EXIT_SUCCESS);
}
```



Il ciclo for prosegue, assegna il valore 5 anche a x[2] e si arriva al punto in cui la condizione non è più verificata, in quanto i non è minore di len

```
void riempi_array(int a[], int len) {
    for (int i = 0; i < len; i = i + 1) {
    a[i] = 5;
    }
}

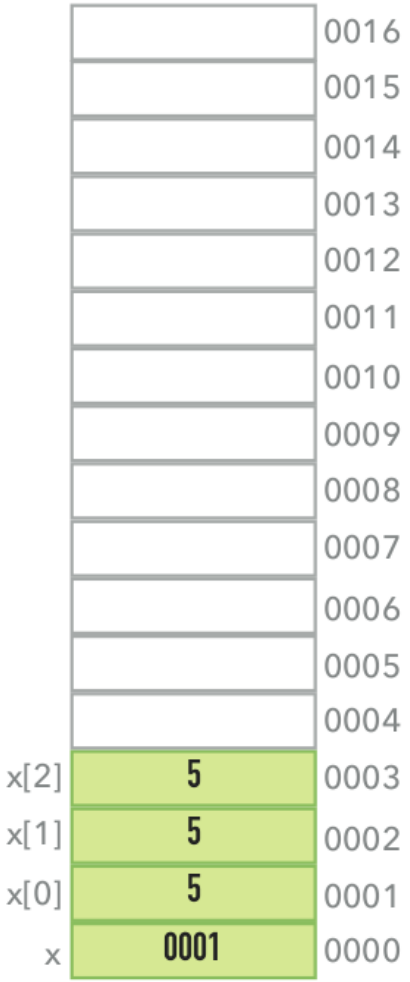
int main(int argc, char * argv[]) {
    int x[3];
    riempi_array(x, 3);
    exit(EXIT_SUCCESS);
}
```



Al termine dell'esecuzione della funzione riempi_array(), il suo record di attivazione viene cancellato, e l'esecuzione torna al main()

```
void riempi_array(int a[], int len) {
    for (int i = 0; i < len; i = i + 1) {
    a[i] = 5;
    }
}

int main(int argc, char * argv[]) {
    int x[3];
    riempi_array(x, 3);
    exit(EXIT_SUCCESS);
}
```



PASSARE ARRAY A UNA FUNZIONE

Riassumendo

- Quando si passano array come argomenti si possono modificare i dati nell'originale e non in una copia
- Questo perché passiamo una copia dell'indirizzo di memoria in cui è memorizzato l'array
- E quindi accediamo ancora al "pezzo di memoria" in cui è memorizzato l'array originale
- **Nota:** Possiamo anche passare array multidimensionali come argomenti di funzioni, ma tutte le dimensioni tranne la prima devono essere specificate:

```
void riempi_array2D(int matrice[][4], int righe) {
    // TODO: write this code
}
```

- **Attenzione:** Non si può restituire un array nel return di una funzione, perché farebbe riferimento a un'area di memoria che si trovava nel record di attivazione ormai eliminato

Campo di visibilità

CAMPO DI VISIBILITÀ (SCOPE) DELLE VARIABILI

- La visibilità (in inglese *scope*), in programmazione, è l'esistenza e la possibilità di richiamare un identificatore, nel nostro caso cioè il nome di una variabile, in un determinato punto del programma

- Una variabile infatti non “esiste” sempre e non è accessibile da ogni parte del codice, ma solo nella sezione del programma detta *scope* della variabile, dove essa è nota

- Esistono quindi regole che dicono che “pezzo di codice” “vede” una certa variabile
  - Una variabile è “visibile” (utilizzabile) nel blocco di codice che la contiene, in pratica dal punto in cui è definita alla chiusura della parentesi graffa che contiene la definizione
  - Nel for le variabili definite nell'inizializzazione (es. la “`i`”...), sono visibili solo nel corpo del `for`
  - Gli identificatori dei parametri di una funzione sono definiti solo nel corpo della funzione, e così le variabili dichiarate all'interno della funzione stessa

- Anche i nomi delle stesse funzioni sono “visibili” (cioè possono essere usati) solo dopo la dichiarazione (prototipo) della funzione stessa

- Si possono dichiarare variabili globali, al di fuori del corpo delle funzioni, visibili in tutte le funzioni che seguono la loro dichiarazione nel codice sorgente. Il loro uso va limitato e in generale si deve evitare di ricorrervi.

CAPIRE IL CAMPO DI VISIBILITÀ, ESEMPI

Consideriamo questa funzione:

```
void scambio(int a, int b) {
    int tmp;
    tmp = a; a = b; b = tmp;
    return;
}
```

Supponiamo di chiamarla in questo frammento di programma:

```
int v = 5; int w = 10;
scambio(v, w);
```

Qual è il valore finale di v e w ?

In questo caso stiamo passando i parametri “per valore”

Appendice alla lezione 3

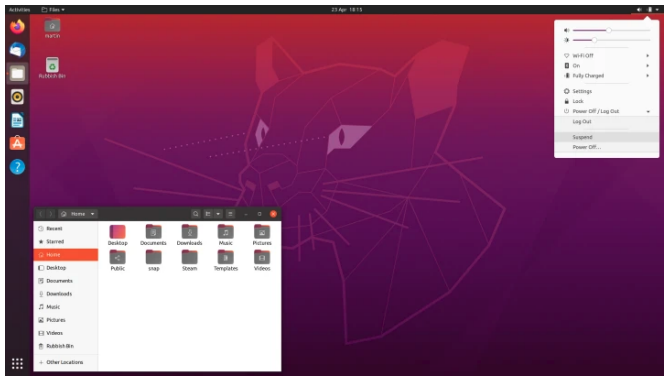
Riga di comando

Canali di comunicazione standard: stdin e stdout

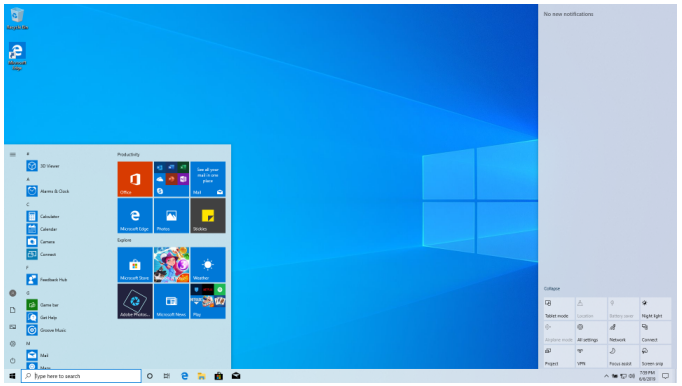
Chiarimenti su `scanf()`

INTERAZIONE CON IL SISTEMA OPERATIVO

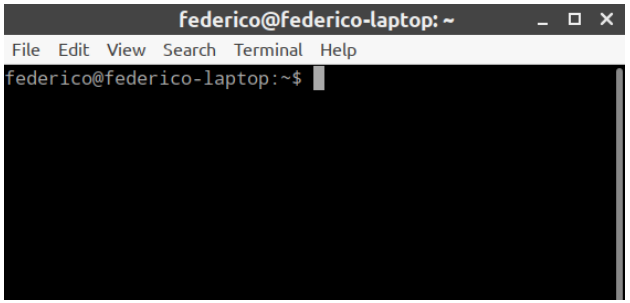
Le persone possono interagire con un calcolatore attraverso un'interfaccia grafica (GUI), con la quale si interagisce tramite mouse, tastiera, touchscreen, tavolette grafiche, touchpad, es.:

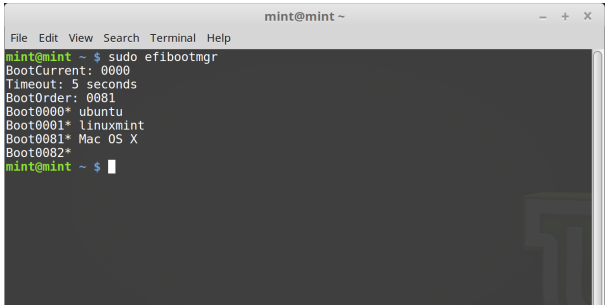


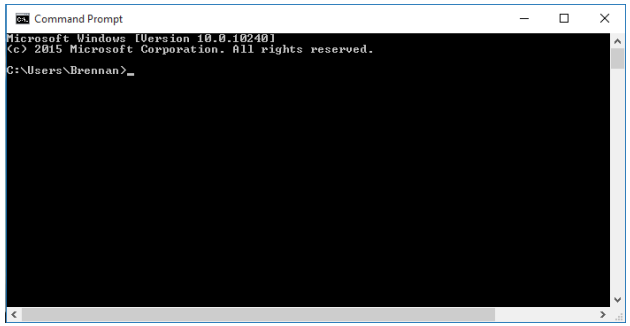




Oppure attraverso un'interfaccia a riga di comando fornita da terminali di testo. In questo caso si usa solo la tastiera per interagire con una shell dei comandi, es:



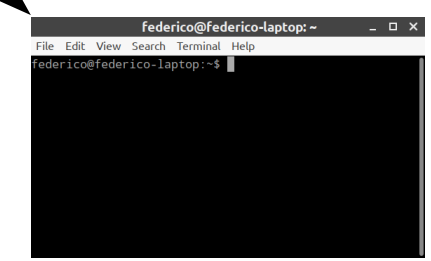




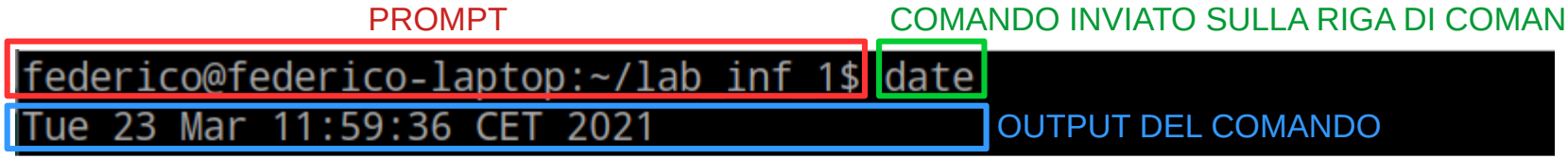
NOTA TERMINOLOGICA

- Per **terminale** originariamente si intendeva un dispositivo fisico dotato di monitor e tastiera che serviva a immettere e leggere informazioni su un computer, eventualmente remoto.
- Dato che possono leggere e scrivere solo testo, sono detti **terminali di testo** o testuali, o text terminal
- Presentano interfacce che servono a immettere comandi, dette quindi **interfaccia a riga di comando**. Un termine alternativo è **console** (es. console di testo o console dei comandi)
- Oggi si fa uso di programmi che emulano un terminale fisico (infatti si chiamano proprio **emulatori di terminale**). Su Linux ce ne sono tantissimi (XTerm è quello tradizionale, ma si trovano vari programmi con il nome “terminal” e simili.
- Il terminale di testo fa uso di una ***shell* dei comandi**, cioè a sua volta un programma che interpreta i comandi che scriviamo sulla **riga di comando** (command line)
- Le shell dei comandi presentano un ***prompt***, cioè una sequenza di caratteri seguiti da un cursore che attende l'immissione dei comandi
- Le shell dei comandi possono essere varie, di norma sui sistemi Linux si usa la shell **Bash**
- Il termine *shell* può indicare un qualunque ambiente in cui si inviano comandi. Per esempio Python ha una sua shell, Gnuplot ha una sua shell, eccetera
- Anche se, come chiarito qui sopra, le parole terminale, emulatore di terminale, terminale di testo, shell dei comandi, console, riga di comando, command line, prompt dei comandi, hanno significati ben precisi, spesso si utilizzano più o meno come sinonimi.
- Se un esercizio richiede di passare delle informazioni a un programma attraverso la riga di comando, si intende che al momento del lancio del programma, quando cioè si scrive sulla shell Bash il nome del programma, esso dovrà essere seguito da parametri che il programma stesso può interpretare. Vedremo che tali parametri sono gli argomenti della funzione main()





LA SHELL DI LINUX: BASH



PROMPT

COMANDO INVIATO SULLA RIGA DI COMANDO

```
federico@federico-laptop:~/lab_inf_1$ date
Tue 23 Mar 11:59:36 CET 2021
federico@federico-laptop:~/lab_inf_1$ date +%s.%N
1616497178.180547006
```

Nome del programma `date` `+%s.%N` Parametro che indica il formato dell'output

```
nomeprogramma opzioni argomenti

gcc -Wall std=c99 -o helloworld helloworld.c
```

Servono a intercettare gli argomenti passati sulla riga di comando

Anticipazione: `int main (int argc, char * argv[])`

UNIX TIME

PROMPT COMANDO INVIATO SULLA RIGA DI COMANDO

```
federico@federico-laptop:~/lab_inf_1$ date
Tue 23 Mar 11:59:36 CET 2021
federico@federico-laptop:~/lab_inf_1$ date +%s.%N
1616497178.180547006
```

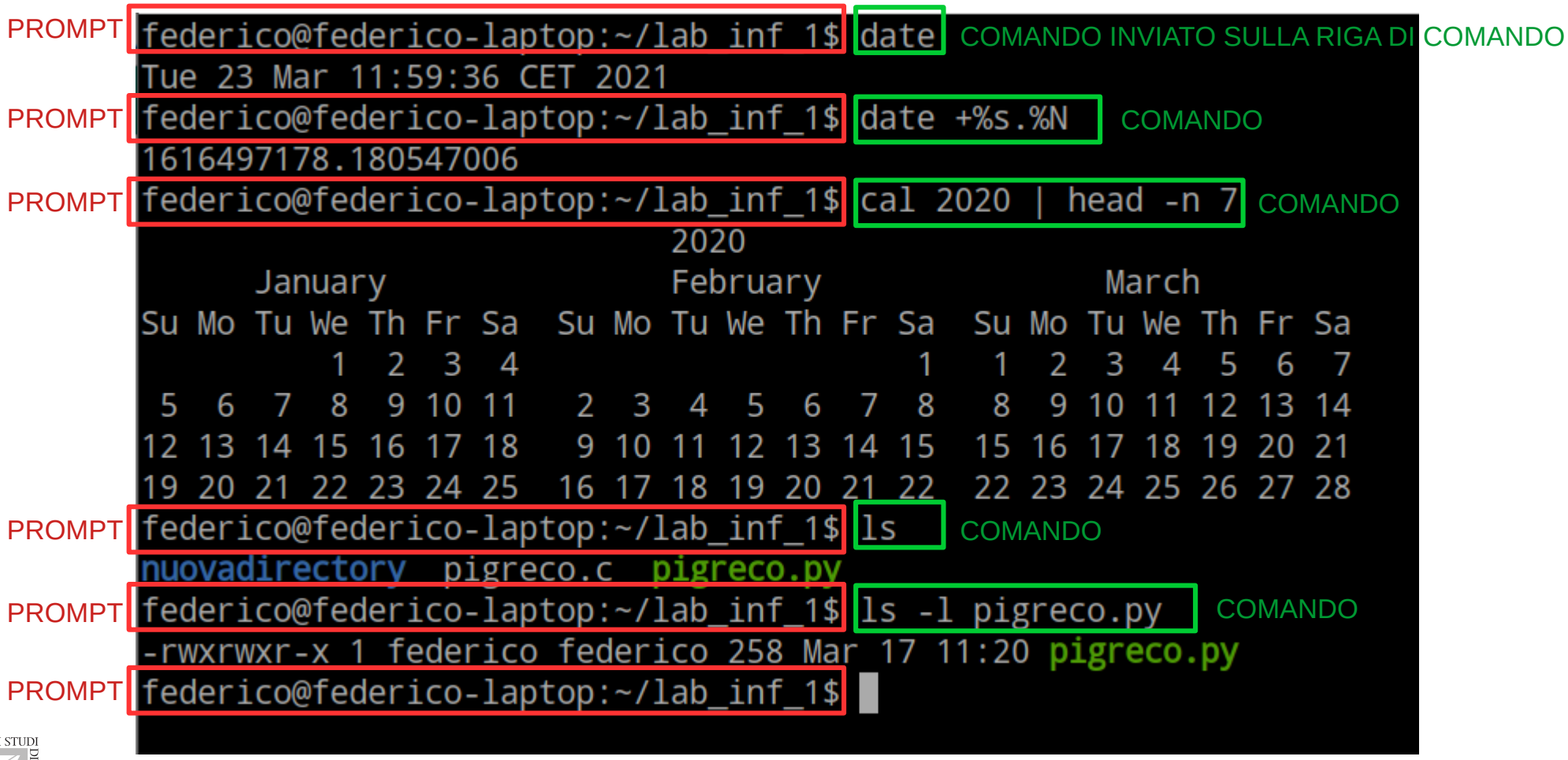
Nota su Unix Time:

- Unix time (o timestamp) o Epoch time o POSIX time, è convenzionalmente il numero di secondi passati dalla mezzanotte UTC dell'1 gennaio 1970
- È utile per inserire una colonna timestamp in una tabella di dati
- È indipendente dai fusi orari
- Per averne una rappresentazione sotto forma di data e ora locali è necessario effettuare una conversione; il sistema operativo offre allo scopo delle funzioni che consentono di effettuarle in entrambe le direzioni

```
federico@federico-laptop:~/lab_inf_1$ date +%s
1616510848
federico@federico-laptop:~/lab_inf_1$ date -d @1616510848
Tue 23 Mar 15:47:28 CET 2021
```

Se rappresentato da un intero a 32 bit, fino a che data si può arrivare? E se fosse a 64 bit?

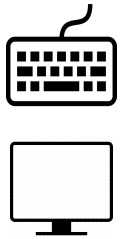
LA SHELL DI LINUX: BASH



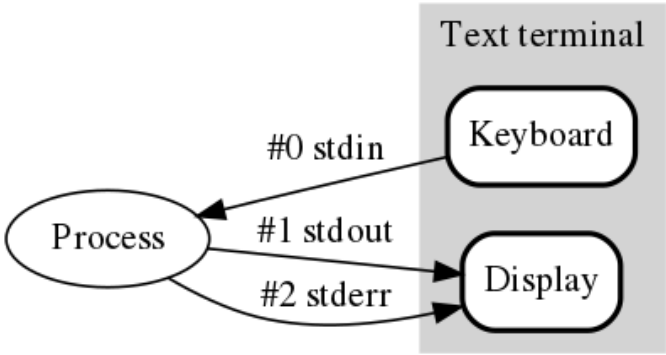
STANDARD INPUT E STANDARD OUTPUT

- I **canali standard** sono dispositivi logici di input e di output che collegano un programma con l'ambiente operativo in cui esso viene eseguito (tipicamente un terminale testuale).

- I tre canali di input/output predefiniti sono detti:



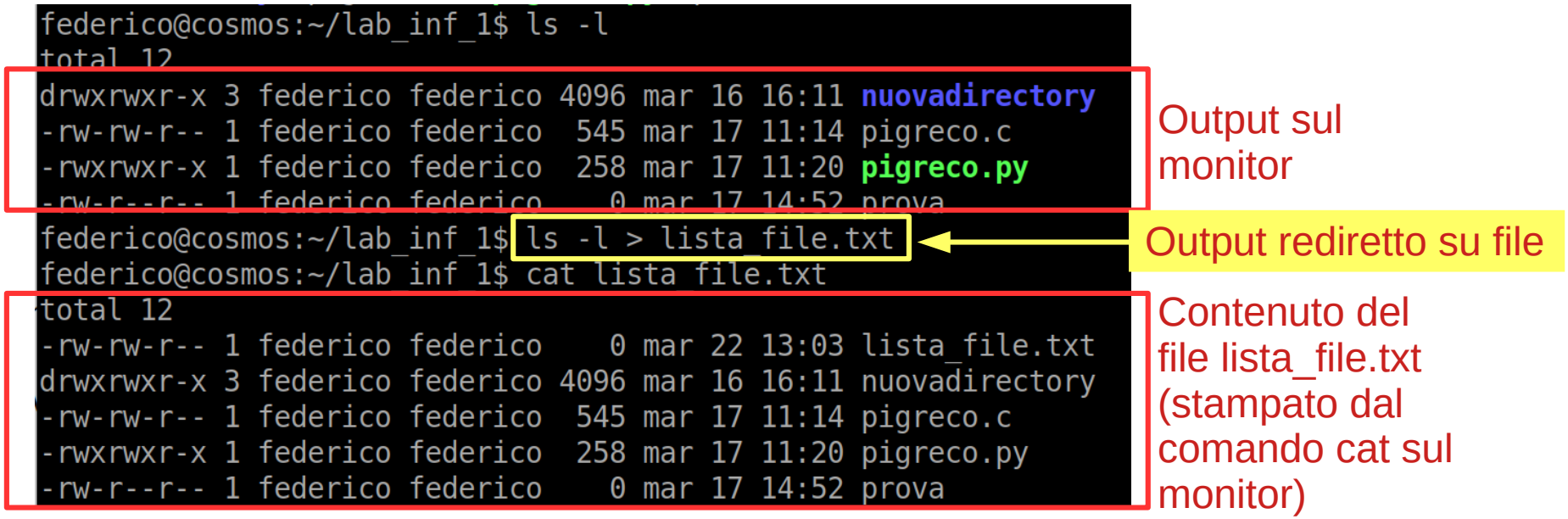
  - **standard input**, abbreviato in stdin, di default la tastiera
  - **standard output**, abbreviato in stdout, di default il monitor
  - **standard error**, abbreviato in stderr, di default il monitor



- Questi canali predefiniti sono disponibili nei sistemi operativi Unix e Unix-like, negli ambienti d'esecuzione del linguaggio C e C++ e nei loro derivativi.

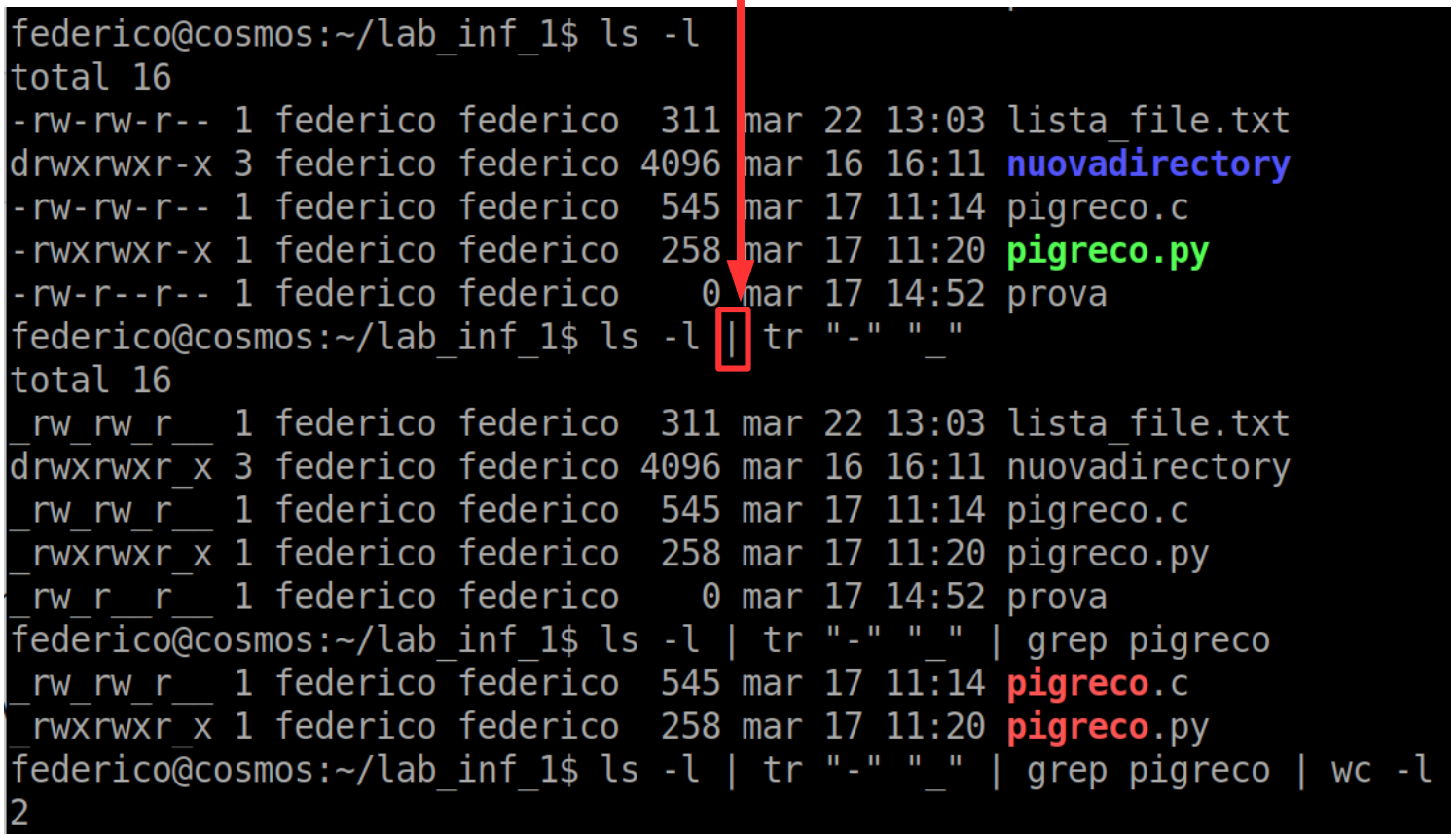
STANDARD INPUT E STANDARD OUTPUT

- I canali standard possono essere **rediretti** attraverso appositi simboli inseriti sulla shell dei comandi, in particolare:
  - Lo stdout può essere inviato in scrittura su un file con **>**, oppure in aggiunta in fondo (append) a un file con **>>**
  - Analogamente, lo stdin può essere letto da un file con **<**



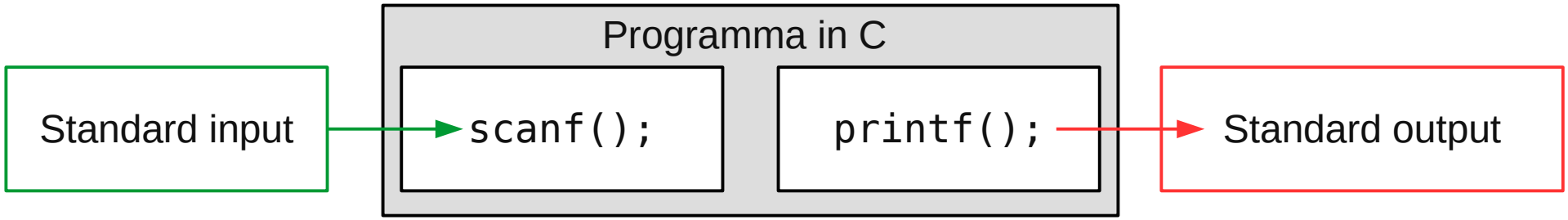
STANDARD INPUT E STANDARD OUTPUT

Lo stdout di un programma può essere collegato allo stdin di un altro programma con il simbolo **|**, detto ***pipe***



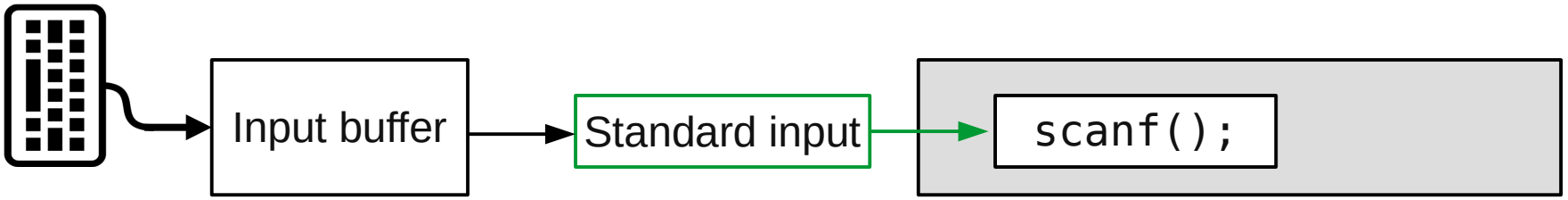
I/O CON I CANALI STANDARD IN C

- I **canali standard** possono essere utilizzati nel linguaggio C usando funzioni fornite dalla libreria stdio.h
- Per esempio si possono usare scanf() e printf()
- Vedremo anche getchar() e putchar()



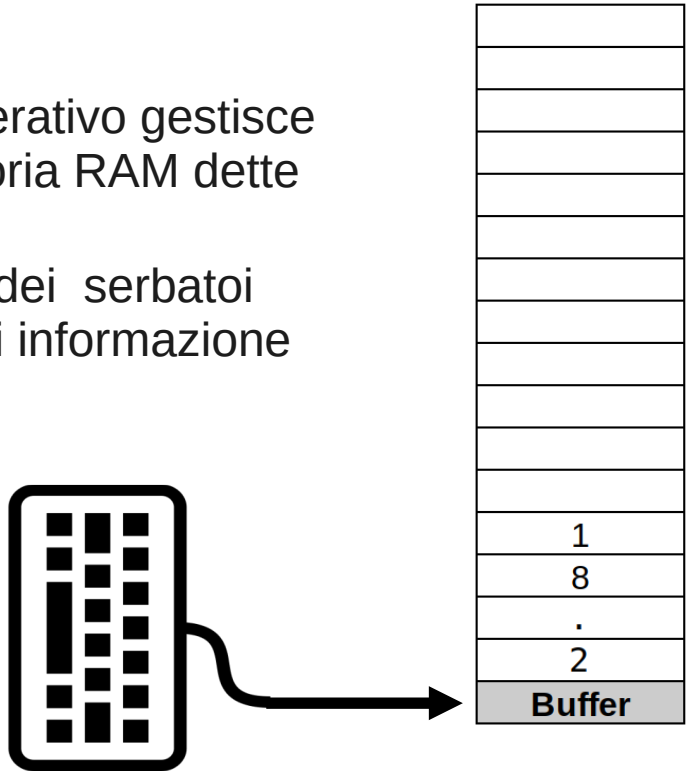
BUFFER DI INPUT E SCANF()

- Il sistema operativo gestisce aree di memoria RAM dette *buffer*
- I buffer sono dei serbatoi temporanei di informazione



BUFFER E SCANF()

- Il sistema operativo gestisce aree di memoria RAM dette *buffer*
- I buffer sono dei serbatoi temporanei di informazione



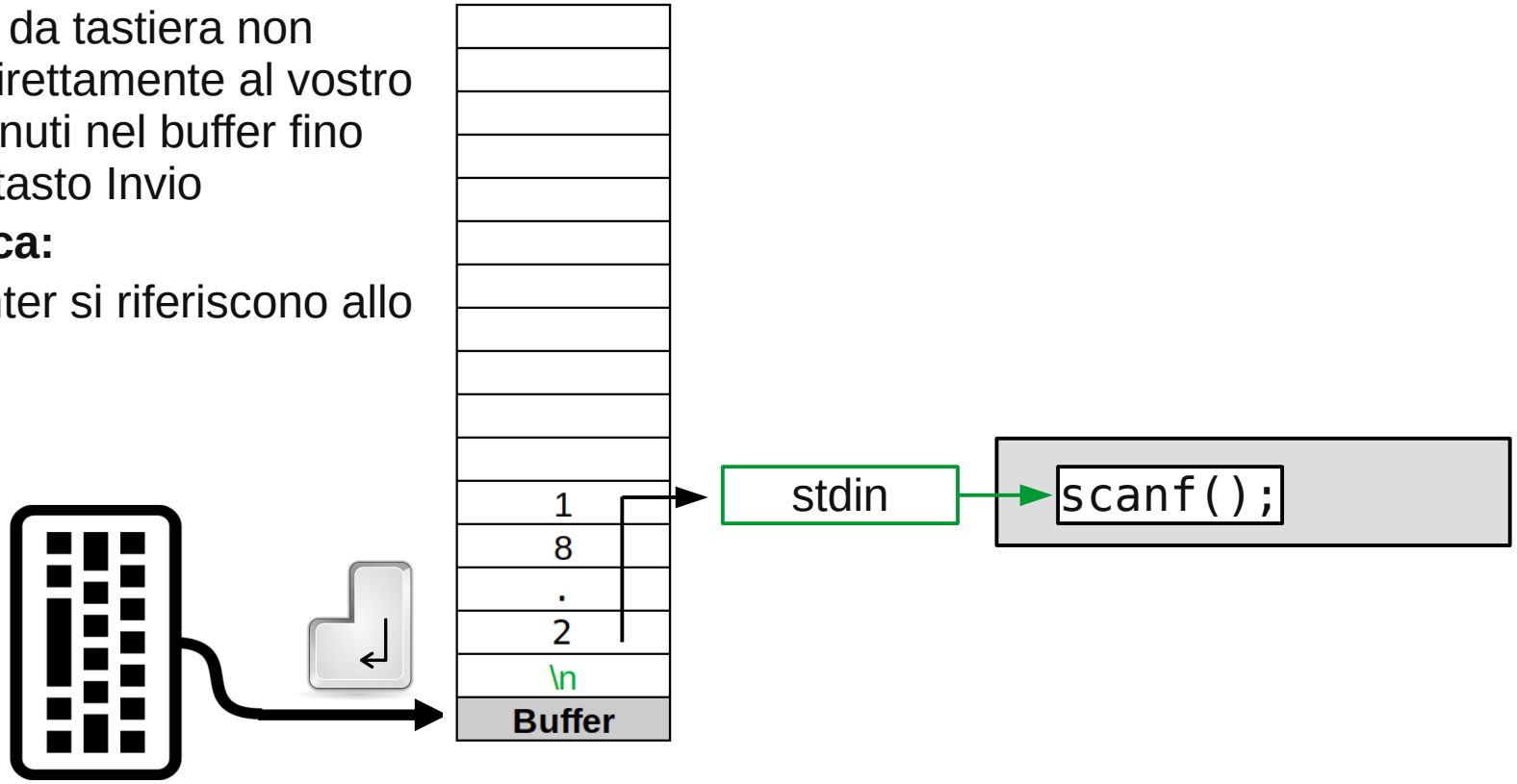
Qui supponiamo che il nostro programma attenda del testo e che sulla nostra tastiera scriviamo 18.2: il sistema operativo prende questi caratteri e li parcheggia temporaneamente in un buffer

BUFFER E SCANF()

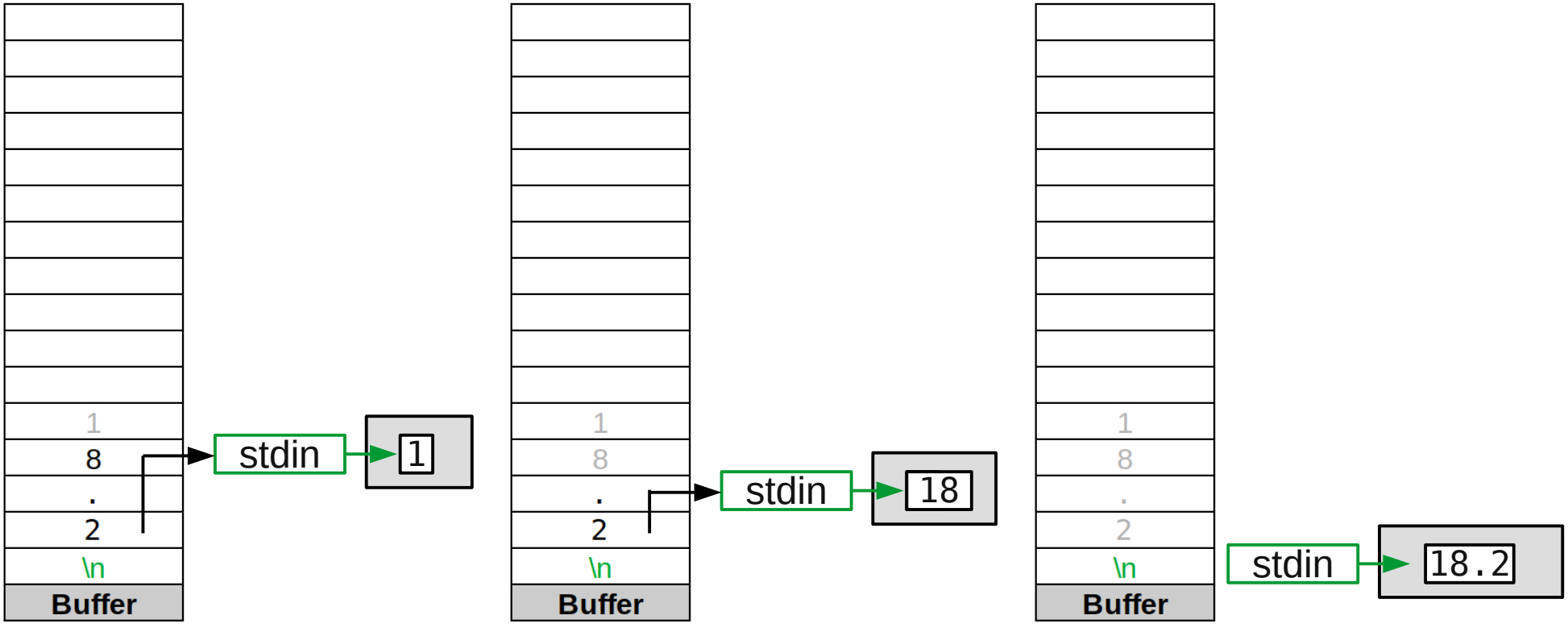
I caratteri immessi da tastiera non vengono passati direttamente al vostro programma, ma tenuti nel buffer fino alla pressione del tasto Invio

Nota terminologica:

Invio, Return, o Enter si riferiscono allo stesso tasto: 

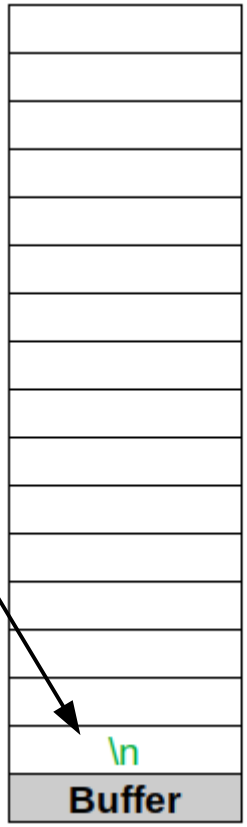


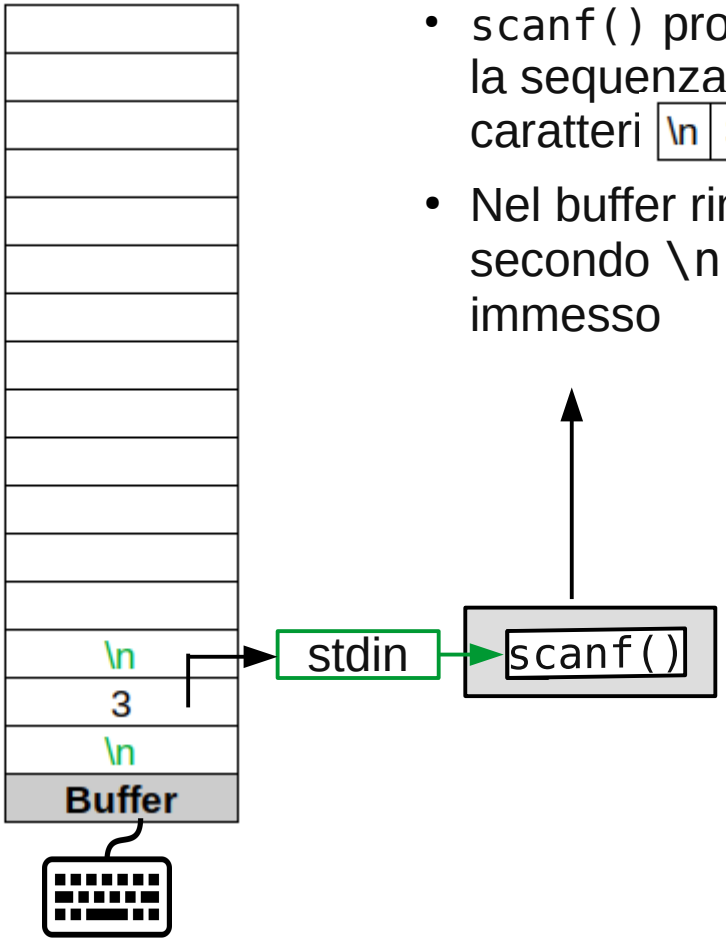
BUFFER E SCANF()



BUFFER E SCANF()

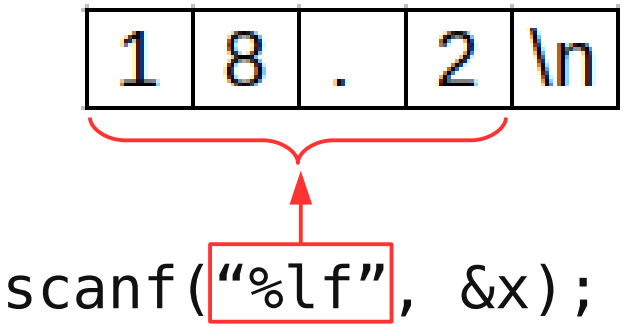
Quando la chiamata a scanf() termina, nel buffer è rimasto un carattere di ritorno a capo (trattato come whitespace da scanf() successivi.





- `scanf()` processerà la sequenza di caratteri | \n | 3 |
- Nel buffer rimarrà il secondo `\n` immesso

LEGGERE LO STDIN CON SCANF()



- 18.2\n sono 5 caratteri, incluso \n
- il formato %lf intercetta 18.2, lo converte in un valore double e lo assegna alla variabile di nome x (che sarà quindi di tipo double)
- il carattere \n viene lasciato nel buffer
- \n (ritorno a capo), spazi bianchi e \t (carattere di tablatura) sono considerati “whitespace”. Tutto il resto è non-whitespace characters
- Fare attenzione se si usano spazi nel formato di scanf(), non inserirli comunque alla fine!

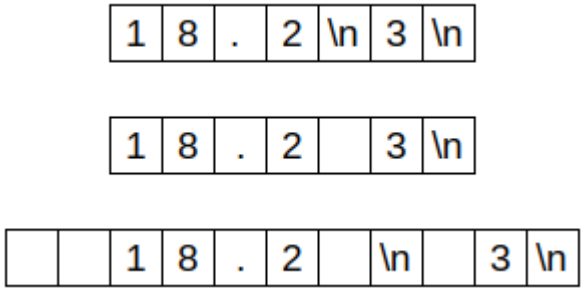
SCANF()

```
double data1;
int data2;
```

```
scanf("%lf %d", &data1, &data2);
```

OPPURE

```
scanf("%lf", &data1);
scanf("%d", &data2);
```



Queste sequenze di immissione vengono convertite nello stesso modo

- il formato “%lf %d”:
  - consuma e scarta qualsiasi whitespace fino al primo non-whitespace
  - legge 18.2 come %lf e lo assegna a data1
  - poi legge e consuma (scarta) qualsiasi whitespace character
  - poi intercetta 3 come %d e lo assegna a data2
- Nel secondo caso succede la stessa cosa, con la differenza che il carattere \n viene “consumato” dall'esecuzione di scanf()

Nota: Posso scrivere i dati in un file ed eseguire il programma redirezionando lo standard input opportuanamente (vedi prossima slide)

REDIREZIONE DI STDIN

```
federico@federico-laptop:~/lab_inf_1$ cat dati.dat

18.3


  3

federico@federico-laptop:~/lab_inf_1$ cat dati.dat | ./test_scanf
18.300000 3.000000
federico@federico-laptop:~/lab_inf_1$ ./test_scanf < dati.dat
18.300000 3.000000
```